Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K4 – Analyse

Analyse the parking problem and identify the objects that a Java program must model.

Discuss:

- The inputs the system receives and the outputs it must produce
- The nouns in the problem that become candidate classes, and the verbs that become methods
- The responsibilities of each candidate class
- The business rules every valid operation must satisfy (for example: a slot cannot hold two vehicles; a ticket number is never reused)
- Any assumptions you make about the facility

You are expected to **justify why each candidate class is needed**, not merely list names.

Alternative Concept and Complexity Analysis

In your design each vehicle is an object. If the vehicles were instead stored as plain Strings in parallel arrays (regNo[], type[], entryTime[]), what would the solution look like? Compare both designs for finding a vehicle by registration number and for adding a new vehicle attribute. Analyse the time complexity of the search and the space used per vehicle in each design.

Evidence Expected

A clear problem analysis with candidate classes, their responsibilities, business rules and stated assumptions.

Solution:

TASK 01 – Understand the Problem and Identify the Objects

1. Problem Analysis

The Campus Smart-Parking System is designed to manage vehicle parking in a multi-level university parking facility. It allows students, staff and visitors to park their vehicles in suitable available slots.

The system supports three types of vehicles: two-wheelers, cars and electric vehicles (EVs). When a vehicle enters, the system checks for an available slot, allocates the slot, generates a unique parking ticket and records the entry time.

When a vehicle leaves, the system releases the parking slot and calculates the parking fee based on vehicle type and parking duration. Students and staff with valid monthly passes can park free of charge, while visitors must pay the applicable fee.

2. Inputs and Outputs

Inputs	**Outputs**
Vehicle registration number	Unique parking ticket
Vehicle type	Allocated floor and slot
Entry time	Recorded entry details
Exit time	Parking duration
User category	Calculated parking fee
Monthly pass validity	Updated slot availability

3. Identification of Candidate Classes

The nouns in the problem statement are identified as candidate classes, while verbs represent the methods or operations.

Candidate class	**Responsibilities**
Vehicle	Stores registration number and vehicle details.

TwoWheeler	Represents two-wheeler vehicles and their parking rate.
Car	Represents cars and their parking rate.
ElectricCar	Represents electric vehicles and charging facilities.
ParkingSlot	Stores slot number and availability status.
ParkingFloor	Manages the parking slots on a particular floor.
ParkingFacility	Manages floors, allocates slots and releases slots.
ParkingTicket	Stores ticket number, vehicle details, entry time and exit time.
User	Stores user details and category.
ParkingPass	Maintains monthly pass validity for students and staff.

Justification:

Each class represents a separate real-world entity or responsibility. For example, Vehicle stores common vehicle information, while ParkingFacility manages the overall parking operation. Separating these responsibilities makes the system easier to maintain and extend.

4. Nouns and Verbs Identification

Nouns (Candidate Classes):

- Vehicle
- Car
- TwoWheeler
- ElectricCar
- ParkingSlot
- ParkingFloor
- ParkingFacility
- ParkingTicket
- User
- ParkingPass

Verbs (Methods):

Verbs	**Corresponding methods**
Enter	vehicleEntry()
Allocate	allocateSlot()
Generate	generateTicket()
Record	recordEntryTime()
Release	releaseSlot()
Calculate	calculateFee()

Validate	validatePass()
Exit	vehicleExit()

5. Business Rules

The following rules must be satisfied by the parking system:

1. A parking slot can accommodate only one vehicle at a time.
2. A vehicle must be allocated only to an available and suitable slot.
3. Every parking ticket must have a unique ticket number.
4. A vehicle cannot enter again with the same registration number while it is already parked.
5. The parking slot must be released when the vehicle leaves.
6. Parking fees depend on vehicle type and parking duration.
7. Students and staff with valid monthly passes are exempted from parking fees.
8. Visitors must pay the applicable parking fee.
9. Electric vehicles must use designated charging slots when charging is required.
10. Invalid vehicle details and invalid parking slot numbers must be rejected.

6. Assumptions

1. Every vehicle has a unique registration number.
2. The parking facility has multiple floors with a fixed number of slots on each floor.
3. Each vehicle occupies only one parking slot.
4. Parking charges are calculated based on completed or started hours according to the pricing rules.
5. A valid monthly pass is checked during fee calculation.
6. All entry and exit times are recorded by the system.
7. The system uses simulated parking data for implementation and testing.

7. Alternative Concept and Complexity Analysis

Alternative: Using Parallel Arrays Instead of Objects

In the object-oriented approach, each vehicle is represented as a separate Vehicle object containing its registration number, type and entry time.

Instead, the system could store vehicle details using three parallel arrays:

```
String[] regNo = new String[100];

String[] type = new String[100];
```

String[] entryTime = new String[100];

All three arrays use the same index to represent one vehicle.

For example:

Index	**regNo[]**	**type[]**	**entryTime[]**
0	TN01AB1111	Car	09:00
1	TN02CD2222	TwoWheeler	09:30
2	TN03EF3333	ElectricCar	10:00

Comparison of Both Approaches

Criteria	**Vehicle Objects**	**Parallel Arrays**
Data storage	Each vehicle is an object	Data stored in separate arrays
Searching	Linear search, O(n)	Linear search, O(n)
Adding a new attribute	Add a field to Vehicle	Create another parallel array
Data management	Easier to manage	Requires matching array indexes
Maintainability	Better	More difficult for large systems

Time Complexity:

- Object-oriented approach: Finding a vehicle by registration number requires checking each vehicle object. Time complexity is O(n).
- Parallel array approach: The registration number array must be searched sequentially. Time complexity is O(n).

Space Complexity:

- Object-oriented approach: O(n) for storing n vehicle objects.
- Parallel array approach: O(n) for storing three arrays containing n entries each.

The object-oriented approach has additional object and reference overhead, while parallel arrays require separate storage for each attribute.

Conclusion:

The object-oriented approach is preferred because it combines related vehicle information into a single object. Adding a new attribute is easier, and the system becomes more readable, reusable and maintainable. Therefore, the object-oriented design is more suitable for the Campus Smart-Parking System.